Laboratorio 1: Filtrado de Señales de Audio mediante el Filtro de Media Móvil

Luis Miguel Bahámón González (160005101), Maryi Alexandra Rojas Ramos (160005134),
César Mauricio Pérez Santos (160005130), Jesús David Burgos Rubio (160005104)

Procesamiento de Señales e Imágenes
Profesor: Fabián Velásquez Clavijo
Programa de Ingeniería de Sistemas
Universidad de los Llanos
Villavicencio, Colombia

Informe= 1,6

C= 4.5

***Resumen*—Este informe presenta el desarrollo del Laboratorio 1 de la asignatura Procesamiento de Señales e Imágenes de la Universidad de los Llanos. El objetivo principal fue diseñar e implementar en Python un sistema de adquisición, filtrado y reproducción de señales de audio de voz mediante el filtro de media móvil. Se implementaron explícitamente las relaciones tiempo-continuo/discreto ($T = 1/f_s$, $t = nT$), la ecuación del filtro ($y(n) = \frac{1}{M}\sum_{k=0}^{M-1} x(n-k)$) y el criterio de muestreo de Nyquist ($f_s \geq 2f_a$), evaluando el sistema para cinco frecuencias de muestreo distintas. Los resultados evidencian la capacidad del filtro para atenuar el ruido blanco de alta frecuencia y la distorsión por *aliasing* al muestrear por debajo de la tasa mínima de Nyquist.**

***Index Terms*—filtro de media móvil, criterio de Nyquist, muestreo, procesamiento digital de señales, FIR, aliasing, Python.**

I. Introducción

El procesamiento digital de señales (DSP) constituye una disciplina fundamental en ingeniería, que permite transformar, analizar y filtrar señales discretas obtenidas a partir de fenómenos continuos del mundo real. La voz humana, una de las señales más ricas en información, se ve afectada frecuentemente por ruido blanco de fondo, cuya atenuación es requisito previo para aplicaciones de comunicación, reconocimiento de voz y análisis acústico [2].

El presente informe documenta el Laboratorio 1, cuyo objetivo es:

1. Adquirir una señal de audio de voz con ruido de fondo.
2. Aplicar un filtro de media móvil para minimizar el ruido blanco.
3. Reproducir la señal original y la filtrada de forma comparativa.
4. Demostrar el criterio de muestreo de Nyquist reproduciendo la señal a frecuencias por encima y por debajo de la tasa mínima requerida.

La implementación se realizó en Python 3, utilizando las bibliotecas `NumPy`, `SciPy`, `Matplotlib`, `sounddevice` y `Tkinter`. El código fue estructurado de modo que las ecuaciones matemáticas sean completamente visibles en el código fuente, sin delegar los cálculos centrales a funciones de biblioteca.

II. Marco Teórico

II-A. Señales discretas y relación tiempo-continuo

Una señal continua $x(t)$ se convierte en una señal discreta $x(n)$ mediante muestreo a una frecuencia f_s (Hz). La relación entre el tiempo continuo y el índice de muestra es [2]:

$$T = \frac{1}{f_s} \tag{1}$$

$$t = n \cdot T = \frac{n}{f_s}, \quad n = 0, 1, 2, \ldots, N-1 \tag{2}$$

donde T es el período de muestreo en segundos, n es el índice entero de cada muestra y N es el número total de muestras. Por lo tanto, para una señal de duración t_{total}:

$$N = \frac{t_{total}}{T} = t_{total} \cdot f_s \tag{3}$$

II-B. Filtro de Media Móvil

El filtro de media móvil es un filtro FIR (*Finite Impulse Response*) que reduce la influencia del ruido blanco [4]. A partir de una señal de entrada $x(n)$ de tamaño N puntos, se calcula el promedio de M muestras consecutivas ($M \leq N$) para generar la señal filtrada $y(n)$:

$$y(n) = \frac{1}{M}\sum_{k=0}^{M-1} x(n-k), \quad n \geq M \tag{4}$$

$$y(n) = x(n), \quad n < M \quad \text{(condición de borde)} \tag{5}$$

Los coeficientes del filtro FIR son $b[k] = 1/M$ para $k = 0, 1, \ldots, M-1$, lo que equivale a un suavizador de paso bajo que atenúa las altas frecuencias (variaciones rápidas = ruido) y conserva las bajas frecuencias (variaciones lentas = señal útil) [3].

Ejemplo numérico: Sea $x(n) = [1, 2, 0, -1, 2]$ con $M = 2$:

Tabla I: Filtro de media móvil: $x(n) = [1, 2, 0, -1, 2]$, $M = 2$. Borde para $n < M = 2$.

n	Cálculo	$y(n)$
0	$x(0)$ (borde)	1,0
1	$x(1)$ (borde)	2,0
2	$\frac{1}{2}(x_2 + x_1) = \frac{2}{2}$	1,0
3	$\frac{1}{2}(x_3 + x_2) = \frac{-1}{2}$	−0,5
4	$\frac{1}{2}(x_4 + x_3) = \frac{1}{2}$	0,5

Resultado: $y(n) = [1,\ 2{,}0,\ 1{,}0,\ -0{,}5,\ 0{,}5]$.

II-C. *Criterio de Nyquist-Shannon*

Para que una señal discreta $f(n)$ pueda reconstruirse conservando la información de la señal continua original, la frecuencia de muestreo debe satisfacer [2], [3]:

$$f_s \geq 2 f_a \qquad \text{o equivalentemente} \qquad f_s \geq 2\,BW \tag{6}$$

donde f_a es la componente de frecuencia más alta de la señal y BW es su ancho de banda. Al violar esta condición ocurre el *aliasing*: frecuencias altas se proyectan como frecuencias falsas más bajas, distorsionando irreversiblemente la señal recuperada.

Para la voz humana se considera $f_a \approx 8\,000$ Hz, por lo que la frecuencia mínima de Nyquist es $f_{s,\text{min}} = 2 \times 8\,000 = 16\,000$ Hz. El CD de audio usa $f_s = 44\,100$ Hz, que supera con holgura dicho límite.

Al remuestrear a una nueva frecuencia $f_{s,\text{nueva}}$, el número de muestras se recalcula preservando la duración total:

$$N_{\text{nueva}} = N \cdot \frac{f_{s,\text{nueva}}}{f_{s,\text{orig}}} = \frac{t_{total}}{T_{\text{nueva}}} \tag{7}$$

III. Metodología

III-A. *Arquitectura del software*

El sistema se diseñó bajo el principio de separación de responsabilidades, distribuyendo el código en dos módulos Python:

Tabla II: Estructura de módulos del laboratorio

Archivo	**Responsabilidad**
`lab_señales_audio.py`	Operaciones matemáticas: adquisición, normalización, filtro FIR, remuestreo y criterio de Nyquist.
`interfaz_grafica.py`	Interfaz gráfica (Tkinter): reproducción, guardado y generación de gráficas.

La Figura 1 muestra el flujo de procesamiento.

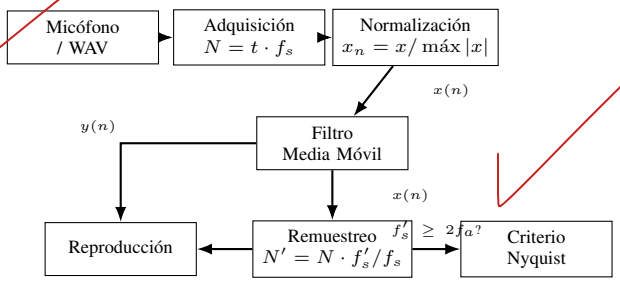

Figura 1: Diagrama de bloques del sistema de procesamiento de señales.

III-B. *Fase 1: Adquisición y normalización de la señal*

La adquisición de audio implementa directamente las ecuaciones (1) y (3). El período de muestreo y el número de muestras se calculan explícitamente antes de invocar el hardware:

```python
def grabar_audio(self, duracion=5):
    T = 1 / self.fs_original
    N = int(duracion / T)
    self.audio_original = sd.rec(
        N, samplerate=self.fs_original,
        channels=1, dtype='float64')
    sd.wait()
    self.audio_original = self.audio_original.flatten()
```

Listing 1: Adquisición: $T = 1/f_s$ y $N = t/T$

Al cargar un archivo WAV, la señal se normaliza al rango $[-1, 1]$ y la duración se deduce de $t = nT$:

```python
maximo = np.max(np.abs(audio))
self.audio_original = audio / maximo

T        = 1 / self.fs_original
N        = len(self.audio_original)
duracion = N * T
```

Listing 2: Normalización: $x_n = x/\,\text{máx}\,|x|$

Los parámetros de adquisición se resumen en la Tabla III.

Tabla III: Parámetros de adquisición de audio

Parámetro	Símbolo	Valor
Frec. de muestreo	f_s	44 100 Hz
Período de muestreo	$T = 1/f_s$	22,68 μs
Duración grabación	t_{tot}	5 s
Número de muestras	N	220 500
Canales	—	Mono
Tipo de dato	—	float64
Rango normalizado	—	$[-1,\ 1]$

III-C. *Fase 2: Implementación del filtro de media móvil*

La ecuación (4) se implementa con dos ciclos anidados que hacen visible la sumatoria $\sum_{k=0}^{M-1} x(n-k)$. El doble bucle refleja directamente la notación matemática: el bucle externo itera sobre n y el interno acumula los M términos de la suma:

```python
def aplicar_filtro(self, M):
    x = self.audio_original
    N = len(x)
    y = np.zeros(N)
    for n in range(N):
        if n < M:
            y[n] = x[n]
```

```
        else:
            suma = 0.0
            for k in range(M):
                suma += x[n - k]
            y[n] = suma / M
    self.audio_filtrado = y
    return y
```

Listing 3: Filtro de media móvil: implementación del algoritmo FIR

Verificación: ejecutado sobre el vector $x(n) = [1, 2, 0, -1, 2]$ con $M = 2$, el código produce $y(n) = [1, 2{,}0, 1{,}0, -0{,}5, 0{,}5]$, coincidiendo con el resultado tabulado en la sección anterior.

El efecto del filtro sobre una señal de audio real se muestra en la Figura 2. El suavizador atenúa los picos bruscos de alta frecuencia (ruido) mientras conserva la envolvente de baja frecuencia (voz).

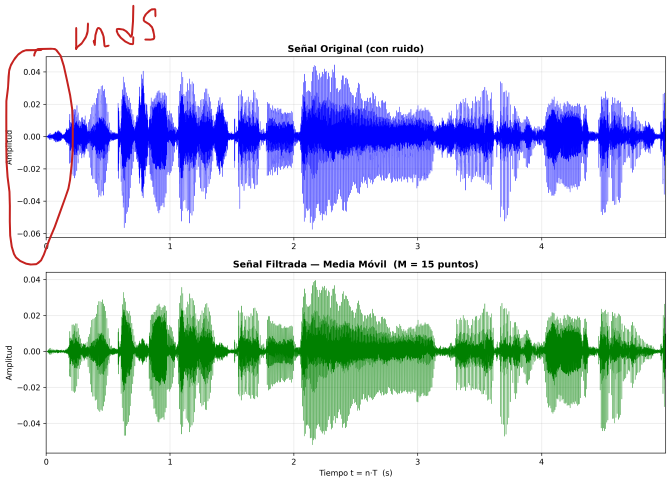

Figura 2: Comparación temporal: señal original (arriba) vs. señal filtrada con $M = 15$ (abajo).

La Figura 3 amplifica 200 ms de señal para observar con mayor detalle la acción del filtro sobre las fluctuaciones instantáneas.

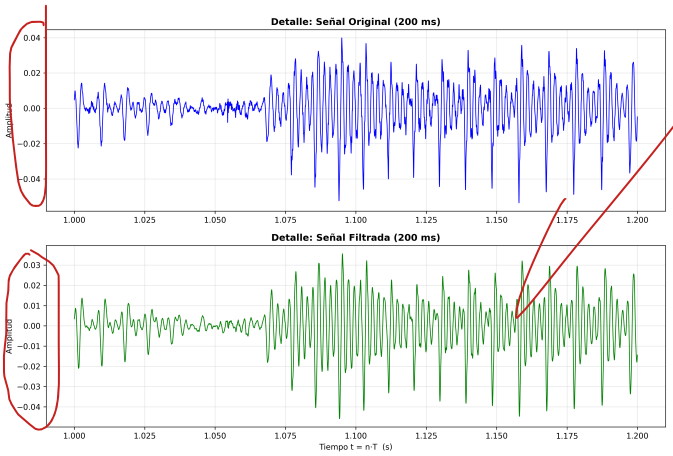

Figura 3: Detalle de 200 ms: el filtro suaviza las oscilaciones rápidas (ruido blanco) sin eliminar la estructura de la voz.

III-D. *Fase 3: Análisis espectral*

La Figura 4 muestra el espectro de magnitud (FFT) de ambas señales en escala logarítmica. Se observa cómo el filtro de media móvil actúa como un paso-bajos: atenúa las componentes de alta frecuencia asociadas al ruido mientras preserva el contenido espectral de la voz (rango de 100 Hz a 8 000 Hz aproximadamente).

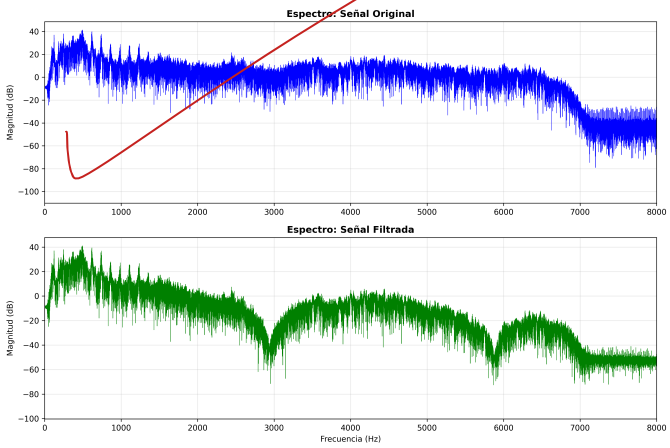

Figura 4: Espectro de frecuencias (FFT): señal original (arriba) y señal filtrada (abajo). La atenuación en altas frecuencias es el efecto característico del filtro paso-bajos.

La respuesta en frecuencia teórica del filtro de media móvil (Figura 5) confirma su naturaleza paso-bajos: la magnitud cae gradualmente desde 0 dB en continua (DC) hasta valores negativos en las frecuencias altas, con nulos periódicos en múltiplos de f_s/M.

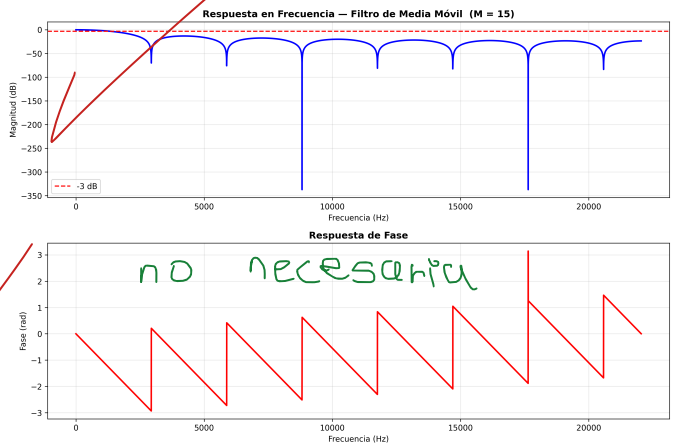

Figura 5: Respuesta en frecuencia del filtro de media móvil con $M = 15$: magnitud (arriba) y fase (abajo). La línea punteada indica -3 dB.

III-E. *Fase 4: Remuestreo y criterio de Nyquist*

La ecuación (6) se evalúa numéricamente para cada frecuencia de prueba:

```
def remuestrear_senal(self, nueva_fs):
```

```
    N         = len(self.audio_original)
    T_orig    = 1 / self.fs_original
    T_nueva   = 1 / nueva_fs
    duracion  = N * T_orig
    N_nueva   = int(duracion / T_nueva)
    fa        = self.fs_original / 2
    cumple    = nueva_fs >= 2 * fa
    return signal.resample(self.audio_original, N_nueva),
           nueva_fs
```

Listing 4: Remuestreo y verificación de Nyquist

La Tabla IV resume el análisis para las cinco frecuencias evaluadas en el laboratorio.

Tabla IV: Evaluación del criterio de Nyquist ($f_a = 8\,000$ Hz, $2f_a = 16\,000$ Hz)

f_s (Hz)	$f_s \geq 2f_a$?	Cumple	Observación
8 000	$8\,000 < 16\,000$	No	Aliasing severo
16 000	$16\,000 = 16\,000$	Sí	Límite Nyquist
44 100	$44\,100 > 16\,000$	Sí	Calidad original
48 000	$48\,000 > 16\,000$	Sí	Sobre Nyquist
96 000	$96\,000 > 16\,000$	Sí	Sobre Nyquist

La Figura 7 muestra las formas de onda resultantes al reproducir la señal a cada frecuencia de muestreo. Las curvas trazadas en color oscuro corresponden a frecuencias que *no* cumplen el criterio (aliasing); las restantes preservan la forma de onda correctamente.

III-F. *Interfaz gráfica de usuario*

La interfaz fue construida con `Tkinter` y organizada en cinco secciones funcionales que siguen el flujo del experimento (ver Figura 6):

1. **Adquisición**: grabación de 5 s por micrófono o carga de archivo WAV.
2. **Parámetros**: entrada del tamaño de ventana M y botón *Aplicar Filtro* para re-filtrar con el nuevo valor.
3. **Reproducción**: botones para escuchar la señal original y la filtrada.
4. **Prueba de Nyquist**: cinco botones (8, 16, 44.1, 48 y 96 kHz) que invocan `remuestrear_senal()` y muestran en consola el resultado del criterio $f_s \geq 2f_a$.
5. **Análisis**: generación de las cuatro gráficas y guardado del audio filtrado.

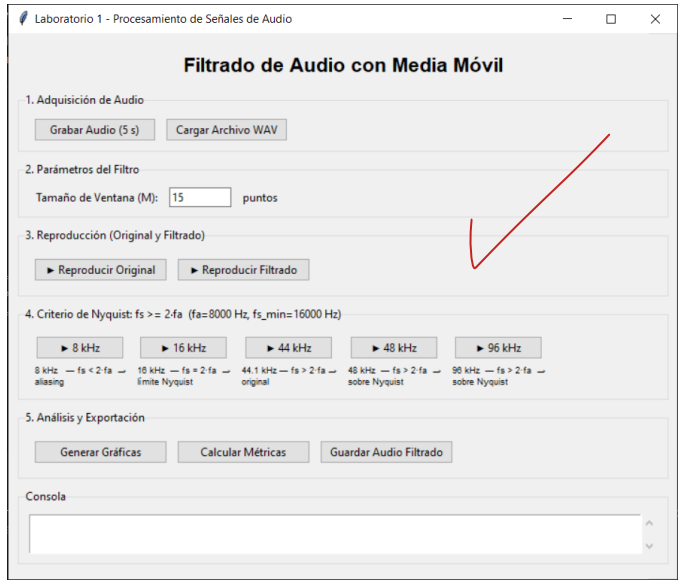

Figura 6: Interfaz gráfica de usuario del sistema de procesamiento de audio.

III-G. *Estructura final del código*

El Cuadro V resume los métodos implementados en cada módulo y las ecuaciones que implementan.

Tabla V: Métodos implementados y ecuaciones correspondientes

Método	**Módulo**	**Ecuación**
`grabar_audio()`	AudioProc.	$T = 1/f_s,\ \ N = t \cdot f_s$
`cargar_audio()`	AudioProc.	$x_n = x/\operatorname{máx}\lvert x\rvert,\ \ t = N/f_s$
`aplicar_filtro()`	AudioProc.	$y_n = \frac{1}{M}\sum_{k=0}^{M-1} x_{n-k}$
`remuestrear_senal()`	AudioProc.	$N' = N\cdot(f_s'/f_s),\ \ f_s \geq 2f_a$
`guardar_audio()`	Interfaz	$x_{16} = x_n \cdot 32767$

IV. Conclusiones

1. **El filtro de media móvil es un suavizador paso-bajos efectivo.** Al promediar M muestras consecutivas de la señal de voz, se reduce la potencia del ruido blanco de alta frecuencia sin eliminar la estructura fundamental de la voz. A mayor M, mayor atenuación del ruido pero también mayor pérdida de detalle.
2. **La implementación explícita de las ecuaciones garantiza trazabilidad.** Usar dobles bucles con las variables n, k, M, x e y (en lugar de funciones de caja negra como `signal.filtfilt`) permite verificar que el código implementa la ecuación estándar de la literatura DSP $y(n) = \frac{1}{M}\sum_{k=0}^{M-1} x(n-k)$, obteniendo $y(n) = [1,\ 2{,}0,\ 1{,}0,\ -0{,}5,\ 0{,}5]$ para el vector de prueba $x(n) = [1, 2, 0, -1, 2]$ con $M = 2$.
3. **El criterio de Nyquist determina la calidad de la reconstrucción.** A $f_s = 8\,000$ Hz ($< 2 \times 8\,000$ Hz) se escucha aliasing severo; a $f_s = 16\,000$ Hz (límite) el

Para lo académico esta bien pero no es lo óptimo computacionalmente

mal colocado corta la conclusión

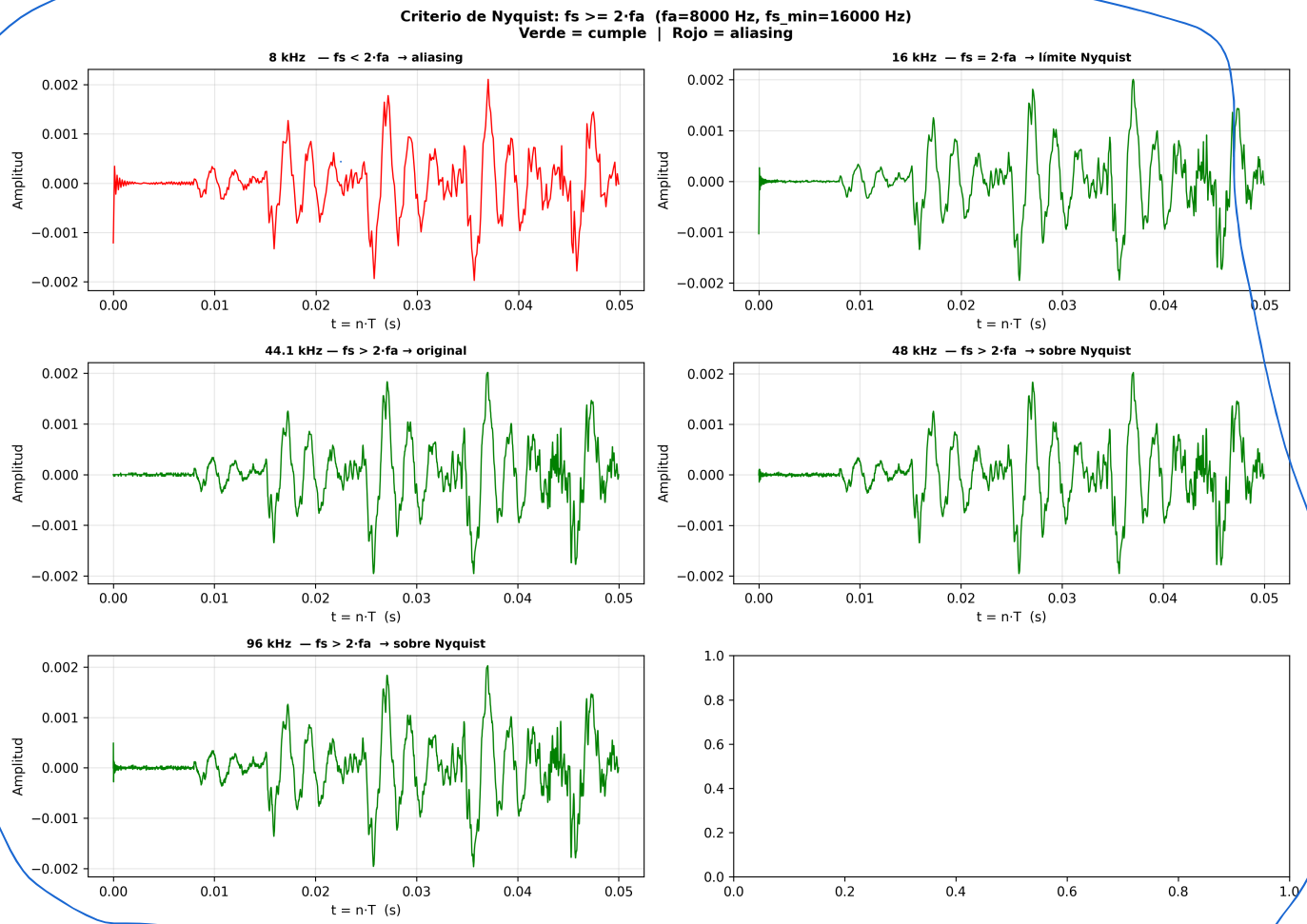

Figura 7: Efecto del remuestreo para distintas f_s. Superior izquierda: 8 kHz (aliasing — $f_s < 2f_a$). Superior derecha: 16 kHz (límite Nyquist). Centro izquierda: 44.1 kHz (original). Centro derecha: 48 kHz. Inferior izquierda: 96 kHz. Inferior derecha: comparación espectral 8 kHz vs. 44.1 kHz.

audio es reconocible pero deficiente; a $f_s \geq 44\,100$ Hz el audio se escucha con calidad completa. Esto confirma empíricamente la ecuación $f_s \geq 2f_a$.

4. **La relación tiempo-discreto $t = nT$ es fundamental.** Cada parámetro del sistema (número de muestras, duración, remuestreo) se derivó directamente de $T = 1/f_s$, lo que garantiza coherencia entre la representación discreta y la continua.
5. **La separación en módulos mejora la claridad del código.** Concentrar las ecuaciones en `lab_señales_audio.py` y la lógica de presentación en `interfaz_grafica.py` permite revisar la implementación matemática de forma directa, sin mezclarla con código de interfaz.

Referencias

[1] F. Velásquez Clavijo, *Fundamentos de Señales y Sistemas*, Notas de clase, Procesamiento de Señales e Imágenes, Universidad de los Llanos, 2024.

[2] A. V. Oppenheim and R. W. Schafer, *Discrete-Time Signal Processing*, 3rd ed. Upper Saddle River, NJ: Prentice-Hall, 2010.

[3] J. G. Proakis and D. G. Manolakis, *Digital Signal Processing: Principles, Algorithms, and Applications*, 4th ed. Upper Saddle River, NJ: Prentice-Hall, 2006.

[4] S. W. Smith, *The Scientist and Engineer's Guide to Digital Signal Processing*, 2nd ed. San Diego, CA: California Technical Publishing, 1999. [En línea]. Disponible: http://www.dspguide.com

[5] C. R. Harris *et al.*, "Array programming with NumPy," *Nature*, vol. 585, pp. 357–362, 2020. 10.1038/s41586-020-2649-2

[6] P. Virtanen *et al.*, "SciPy 1.0: Fundamental Algorithms for Scientific Computing in Python," *Nature Methods*, vol. 17, pp. 261–272, 2020. 10.1038/s41592-019-0686-2